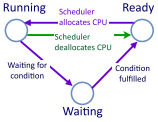
CONCEPTS OF CONCURRENCY

Motivation: Word (GUI), Moore's Law end → Multi-Core Era, scale horizontal
Multi-core: a processor chip containing multiple cores (processing units)
Caches: L1: inside core, L2: inside core, sometimes shared between some cores, L3: on CPU chip, shared between all cores
Parallelism: Subprograms run simultaneously on multiple cores, goal: faster programs
Concurrency: interleaved execution of programs (time-slicing or parallel), goal: simpler programs + non blocking GUI
Process: **Program under Execution**, Heavyweight, own address space, OS needs process context
Thread: **Parallel sequence** in process. Lightweight, same address space, separate stack and registers

THREAD SCHEDULING



core sharing: Run more threads than cores, On waiting: Release core for other ready threads. (Oversubscription)
scheduler: "threads are on mercy of scheduler"
Context switch: **Synchronous** Thread waiting for condition queues itself and gives core free voluntary
Asynchronous context switch due to an external event (hardware or timer interrupt / time-slice, higher-priority thread), prevents thread from permanently occupying.
Cooperative Multi-Tasking (scheduling): Threads must explicitly initiate context switches, scheduler can't interrupt.
Preemptive Multi-Tasking: scheduler can asynchronously interrupt thread via timer interrupt
Time-Sliced Scheduling (preemptive): Each thread has the processor for maximum time interval

JVM THREAD MODEL

Java Virtual Machine (JVM): is a process, Java = single processing system
threads: main thread which calls the main function, programmer can start more threads, Subsystems / Runtime System also start their own threads (e.g. RMI, AWT, Garbage Collector)
Termination: after all (not daemon) threads are finished or System.exit/Runtime.exit
Thread class: override the `run` method of the Runnable interface with the code to be executed in parallel.

create + start a thread
```
// explicit Runnable implementation
class myRunnable implements Runnable {
  @Override
  public void run() {
    /* thread behavior */
} }
var myT = new Thread(new myRunnable());

// named function
var myT = new Thread(() →
namedFunction());

// anonymous function (lambda)
var myT = new Thread(() → {
  /* thread behavior */
}); // C# lambda lambda with ⇒ arrow
```

Multi-Thread Examples Java
```
// no synchronization
public static void main(String[] args) {
  var a = new Thread(() → multiP("A"));
  var b = new Thread(() → multiP("B"));
  a.start(); b.start();
System.out.println("main finished");
} // varying result, non-deterministic scheduler

// with synchronization (a and b from above):
System.out.println("Threads start");
a.start(); b.start(); // ...
a.join(); b.join();
System.out.println("Threads joined");
```

Thread Java & .NET
`t.start()` == `t.Start()` : Start the thread (not run!)
`t.join()` == `t.Join()` : wait till thread finished, timeout: `join(ms)`,
`t.Sleep(ms)` == `t.Sleep(ms)` : join+sleep → Timed-Waiting state,
`t.wait()` : gives processor+lock free, call in synchronized block.
`t.yield()` : scheduler can release processor, to Ready state, does not release lock
`t.setDaemon(true)` == `isBackground = true` :
`t.interrupt()`: for cooperative canceling. Use when Cancel-Policy of threads is known (know what thread does after it was interrupted)
`InterruptedException`: thrown on `interrupt()` while `sleep`/`wait`/`join`.
`t.interrupted()` `t.isInterrupted()`: check interrupted flag
`Thread.currentThread()`: static method that returns `Thread`, reference to current executing thread instance

Java Thread States:
`Blocked` *(blocked and waiting for a monitor lock)* , `New` *(not yet started)* , `Runnable` *(Ready to run or running)* , `Terminated`, `Timed_Waiting` *(waiting with a specified waiting time* `Thread.sleep(ms)`/ `Thread.join(ms)`*)*, `Waiting`

MONITOR SYNCHRONIZATION

Non-determinism: threads run interleaved or parallel
Synchronization (=Restriction of concurrency): for deterministic behavior.
Communication between threads: Sharing access to fields and the objects they refer to. Efficient, but problems: **Thread interference** and **memory consistency errors**.
Critical Section: code must be executed by only 1 thread at a time. Implementation with **mutual exclusion**.

```
class BankAccount {
  private int balance = 0;
  public void deposit(int amount){
    // enter critical section
    synchronized(this) {
      this.balance += amount;
    } // exit critical section
  }
}
```

happens-before relationship: result is same as if executed synchronously (visibility + threadsafe)

Java synchronized (Monitor)
Guarantees **memory consistency**, **happens-before relationship** and **reentrancy**. Every object has a **Lock** *(Monitor-Lock)* . Maximum 1 thread can acquire it. Entry of `synchronized` method acquires lock of the object, exit releases it. `public synchronized void deposit(int amount) { this.balance += amount; }` synchronized is used within method, specify object to lock. `synchronized(this) { this.balance++; }` (unhandled exceptions exit sync. block)

Monitor Lock

Monitor for **internal mutual exclusion**. Only one thread operates at a time in Monitor, has **fairness-problems** and **no shared locks**.

Recursive Lock: thread can acquire same lock through recursive calls, free by last release
Busy Wait: is repeatedly checking a condition without giving CPU free
Wakeup signal: Signalling a thread in Monitor (inner waiting room). `notify()` and `notiyAll` `notify()`: signals any waiting thread. Turnstile
One-In/One-Out: change applies to only one thread, Only a single waiting thread can continue
Uniform Waiters: Only 1 semantic condition for every thread, condition interests every waiting Thread
`notifyAll()`: wakes up all threads (from inner waiting room to outer waiting room to wait for entry in monitor)
signal and continue: signaling (notifying) thread releases only after leaving synchronized, awakened thread comes to outer waiting room.
`IllegalMonitorStateException`: thrown if `notify`/`notifyAll`/`wait` used outside synchronized.
spurious Wakeup in Java: threads can wakeup from `wait` without a reason

```
// Java
class BankAccount {
  private int balance = 0;
  public synchronized void withdraw
      (int amount)
      throws InterruptedException {
    while (amount > balance) { // not if
      wait(); // wait on a condition
    }
    balance -= amount;
  } // release / leave monitor
  public synchronized void deposit
  (int amount) {
    balance += amount;
    notifyAll();
} }
```

```
// C# .NET
class BankAccount {
  private decimal balance;
  private object syncObject = new();
  public void Withdraw(decimal amount) {
    lock (syncObject) {
      while (amount > balance)
        Monitor.Wait(syncObject);
      balance -= amount;
  }}
  public void Deposit(decimal amount) {
    lock(syncObject) {
      balance += amount;
      Monitor.PulseAll(syncObject);
} } }
```

HAZARDS AND CONDITIONS OF CORRECTNESS

DR + RC = Erroneous behaviour, **No DR + RC** = Erroneous behaviour, **No RC + DR** = Program works correctly, but formally incorrect, **No RC + No DR** = Correct behaviour
Race Conditions (safety): **Insufficiently synchronized access to shared resources**. The **order of events** affects the **correctness** of the program. Leads to **non-deterministic behavior**.
Race Condition without data race: critical section is not protected. Data Race is eliminated using synchronization, but there is no synchronization over larger blocks, so race conditions are still possible *(i.e. non-atomic incrementing)* .
Data Race: Two threads in a single process **access the same variable** concurrently without synchronization, at least one of them is a **write access**.

THREAD SAFETY

Dispensable cases in synchronization: **Immutable Classes** *(Declaring all fields private and final and don't provide setters)* , **Read-only Objects** *(Read-only accesses are thread-safe)*
Confinement: Object belongs to only one thread at a time. **Thread Confinement** *(Object belongs to only one thread)* , **Object Confinement** *(Object is encapsulated in other synchronized objects)*
Thread safe: A data type or method behaves correctly, when same behaviour when used from multiple threads as if running in a single thread
Thread Safety: Avoidance of Data Races (and for parallelism correctness also no deadlock). **No sharing intended**: give each threada private copy of the data. **Sharing is important**: provide explicit synchronization.

DEADLOCKS (SAFETY)

Happens when threads lock each other out, prohibiting both from running. Programs with potential deadlock are not considered correct. Threads can suddenly block each other.

```
// Thread 1
synchronized(listA) {
  synchronized(listB) {
    listB.addAll(listA); }}

// Thread 2
synchronized(listB) {
  synchronized(listA) {
    listA.addAll(listB); }}
```

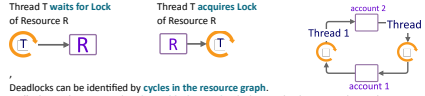
Both threads in this scenario have **locked each other out**, the program cannot continue.

Livelock: Threads have blocked each other permanently, but still execute wait instructions and therefore consume CPU during deadlock.

```
// Thread 1
b = false; while(!a) { } ... b =
true;

// Thread 2
a = false; while(!b) { } ... a =
true;
```

Resource Graph:



,
Deadlocks can be identified by **cycles in the resource graph**.
Deadlock Avoidance: Introduce **linear blocking order**, lock nested only in ascending order. Or use **coarse granular locks** *(e.g. block the whole Bank to block all accounts)*

STARVATION (LIVENESS)

Thread never gets chance to access a resource. **Avoidance**: Use fair synchronization constructs. *(Aging, Enable fairness in previous synchronization constructs. Monitor and Thread priorities have a fairness problem.)*

MEMORY MODELS

Lock-Free Programming: **Correct** concurrent interactions **without using locks**. Use guarantees of memory models. Goal is **efficient synchronization**.
Problems: Memory accesses are seen in **different order** by different threads, except when **synchronized** and at **memory barriers** *(weak consistency)* . Optimizations by compiler, runtime system and CPU. Instructions are reordered or eliminated by optimization.
Memory model: Part of language semantics, there exist different models: **sequential consistency (SC)** *(Order of execution cannot be changed. Too strong a consistency model)* and the **Java Memory Model** *(a "weak" memory model)* .

JAVA MEMORY MODEL (JMM)

Interleaving-based semantics. Minimum warranties: **Atomicity, Visibility and Ordering**.

Atomicity
An **atomic** action is one that happens **all at once** *(So no thread interference)* . Java guarantees that read/writes to primitive data types up to 32 Bit, Object-References *(strings etc.)* and long and double *(with `volatile` keyword)* are atomic. **A single read/write is atomic**. Atomicity does **not imply visibility**.

Visibility
Guaranteed visibility between threads. **Lock Release & Acquire** *(Memory writes before release are visible after acquire)* , **`volatile` Variable** *(Memory writes up to and including the write to volatile variables are visible when reading the variable)* , **Thread/Task-Start and Join** *(Start: input to thread; Join: thread result)* , **Initialization of `final` variables** *(Visible after completion of the constructor)* , **`final` fields**.

Java Ordering
Java Happens Before: "Happens before" defines the **ordering and visibility guarantees** between actions in a program. It ensures that changes made by one thread become visible to others. An **unlock** of a monitor **happens-before** every subsequent lock of that same monitor.
Java Ordering Guarantees: Writes before Unlock → reads after lock, `volatile` write → `volatile` read, Partial Order. Synchronization operations are never reordered. *(Lock/Unlock, volatile-accesses, Thread-Start/Join.)* .

SYNCHRONIZATION IN MEMORY MODEL (LOCK-FREE, ATOMIC)

Rendez-Vous: Primitive attempt to synchronize threads.

```
// Java
volatile boolean a = false, b = false;
// Thread 1
a = true; while( !b ) { ... }
// Thread 2
b = true; while( !a ) { ... }
```

```
// C# .NET
volatile bool a = false, b = false;
// Thread 1
a = true; Thread.MemoryBarrier();
while (!b) { ... }
// Thread 2
b = true; Thread.MemoryBarrier();
while (!a) { ... }
```

No reordering because a and b are `volatile`.

Spin-Lock with atomic Operation:
```
public class SpinLock {
  private final AtomicBoolean locked = new AtomicBoolean(false); // unlocked
  public void acquire() { while( locked.getAndSet(true) ) {...} }
  public void release() { locked.set(false); }
}
```
Java Atomic Classes: Classes for boolean, Integer, Long, References and Array-Elements. Different kinds of atomic operations, `addAndGet()`, `getAndAdd()` etc.
Java Operations on atomic data classes: `boolean getAndSet(boolean newValue)`
Atomically sets to the given value and returns the previous value.
`boolean compareAndSet(boolean expect, boolean update)`
Sets `update` only when read value is equal to `expect`. Returns true when successful.
Optimistic Synchronization: *(Read old value and then compare before writing if value is still the same. If not, retry)*
`do { oldV = v.get(); newV = result; } while(!v.compareAndSet(oldV, newV));`
Lambda-Variants: `AtomicInteger s = new AtomicInteger(2); s.updateAndGet(x → x * x);`

C# Lock-free bank account
```
public bool Withdraw(int amount) { int localB;  while (true) {
    localB = Balance; /* Balance ⇒ Volatile.Read(ref _balance); */
    Thread.MemoryBarrier(); if (amount > localB) { return false; }
    // if original _balance equals original local balance, the operation worked:
    if (Interlocked.CompareExchange(ref _balance, localB - amount, localB)
        == localB) { return true; }
} /* while */ }
```

.NET MEMORY MODEL

Differences to JMM: **Atomicity** *(long/double not atomic with volatile)* , **Ordering and Visibility** *(only half and full fences)* . **Atomic Instructions** with the `Interlocked` class

Half Fence (Volatile)
Reordering in one direction still possible. **Volatile Write:** Release semantics *(Preceding memory accesses NOT moved below it, but later operations can be executed before write)* .

Volatile Read: Acquire semantics *(Subsequent memory accesses NOT mpved above it, but previous operations can be executed after the read)*

Full Fence (Memory Barrier)
Disallows reordering in both directions. `Thread.MemoryBarrier();`

SPECIFIC SYNCHRONIZATION PRIMITIVES

.NET SYNCHRONIZATION PRIMITIVES

C# has no lock and condition and no fairness flag

.NET Monitor
– use sync object: `private object sync = new(); lock(sync){ ... }.`
– `Monitor.Wait(sync)`, `Monitor.PulseAll(sync)`, `Monitor.Pulse(sync)`.
– Wait in Loop: without loop: overtaking problem (in signal and continue).

Uses fair FIFO-Queue. **Locks:** No fairness flag, no Lock & Condition. **Additional:** `ReadWriteLockSlim` for Upgradeable Read/Write, Semaphores can also be used at OS level, Mutex. Collections are **not** Thread-safe.

SEMAPHORE

Allocation of a limited number of free resources. Is in essence a **counter**. If a resource is **acquired**, `count--`, if a resource is **released**, `count++`. Can wait until resource becomes available. Can also acquire/release multiple permits at once atomically.

```
public class Semaphore {
  private int value; public Semaphore(int initial) { value = initial; }
  public synchronized void acquire() throws InterruptedException {
    while (value <= 0) {
      wait(); } value--; }
  public synchronized void release() { value++; notify(); } }
```

General Semaphore: `new Semaphore(N)` *(Counts from 0 to N for limited permits to access a resource)*
Binary Semaphore: `new Semaphore(1)` *(Counter 0 or 1 for mutual exclusion, open/locked)*
Fair Semaphore: `new Semaphore(N, true)` *(Uses FIFO Queue aging for fairness. Slower than non-fair variant.)*
Semaphores are **powerful**, any synchronization can be implemented. But relatively **low-level**.

```
class BoundedBuffer<T> {
  private Queue<T> queue = new LinkedList<>();
  private Semaphore upperLimit = new Semaphore(Capacity, true); // how many free?
  private Semaphore lowerLimit = new Semaphore(0, true); // how many full?
  public void put(T item) throws InterruptedException {
    upperLimit.acquire(); // free places - 1
    synchronized (queue) { queue.add(item);}
    lowerLimit.release(); } // full places + 1
  public T get() throws InterruptedException {
    T item;
    lowerLimit.acquire(); // full places - 1
    synchronized (queue) { item = queue.remove(); }
    upperLimit.release(); // free places + 1
    return item; }}
```

LOCK & CONDITION

Monitor with **multiple waiting lists** and conditions. Independent from Monitor locks.
Lock-Object: Lock for entry in the monitor *(outer waiting room)*
Condition-Object: Wait & Signal for a specific condition *(inner waiting room)* .
ReentrantLock: Class in Java, **alternative to `synchronized`**. Allows multiple locking operations by the same thread and supports nested locking *(Thread is able to re-enter the same lock)* .
Condition: Factors out the Object monitor methods *(`wait`, `notify` and `notifyAll`)* into distinct objects to give the effect of having multiple wait-sets per object, by combining them with the use of arbitrary `Lock` implementations. A **Condition replaces the use of the Object monitor methods**.
`condition.await()`: Throws an `InterruptedException` if the current thread has its interrupted status set on entry to this method or is interrupted while waiting *(`finally` frees the lock in case of interrupt)* . *`myThread.interrupt()`*

Buffer with Lock & Condition
```
class BoundedBuffer<T> {
  private Queue<T> queue = new LinkedList<>();
  private Lock monitor = new ReentrantLock(true); // fair queue
  private Condition nonFull = monitor.newCondition();
  private Condition nonEmpty = monitor.newCondition();
  ...
  public void put(T item) throws InterruptedException {
    monitor.lock(); // Lock queue
    try { while (queue.size() == Capacity) { nonFull.await(); }
      queue.add(item); nonEmpty.signal(); } finally { monitor.unlock(); }
  } // signalAll() if uniform waiters
  public T get() throws InterruptedException {
    monitor.lock(); // wait for queue to be filled & signal to other queue
    try { while (queue.size() == 0) { nonEmpty.await(); }
      T item = queue.remove(); nonFull.signal(); return item;
    } finally { monitor.unlock(); } // always release lock, even after Exception
}}
```

READ-WRITE LOCK

Mutual exclusion is **unnecessary for read-only** threads. So one should allow parallel reading access, but implement mutual exclusion for write access.

Parallel	Read	Write
Read	Yes	No
Write	No	No

```
ReadWriteLock rwLock = new ReentrantReadWriteLock(true); // true for fairness
rwLock.readLock().lock(); // shared Lock
// read-only accesses here
rwLock.readLock().unlock(); // release shared lock
rwLock.writeLock().lock(); // exclusive Lock
// write (and read) accesses here
rwLock.writeLock().unlock(); // release exclusive lock
```

COUNT DOWN LATCH

Synchronization with a counter that can only **count down**. Threads can wait until counter ≤ 0, or they can count down. The Latches can only be used once.

```
var ready = new CountDownLatch(N);   var start = new CountDownLatch(1);
ready.countDown(); // wait for N cars → ready.await(); // wait 4 all cars ready
start.await(); // await race start <--- start.countDown(); // start the race
```

CYCLIC BARRIER

Meeting point for fixed number of threads. Threads wait **until everyone arrives**. Is **reusable**, threads can synchronize in multiple rounds at the same barrier *(Simplifies example above)* .

```
var start = new CyclicBarrier(N); start.await(); // all cars race as they're here
```

EXCHANGER

Rendez-Vous: Barrier with **information exchange** for 2 parties. The Exchanger blocks until another thread also calls `exchange()`, returns argument x of the other thread.

```
var exchanger = new Exchanger<Integer>();
for (int count = 0; count < 2; count++) { // odd number of exch.: last one blocks
  new Thread(() → { for (int in = 0; in < 5; in++) {
      try {
        int out = exchanger.exchange(in); // exchange "in" data
        System.out.println(Thread.currentThread().getName() + " got " + out);
      } catch (InterruptedException e) { }
} /* inner for loop */ }).start(); /* thread */ } /* outer for loop */
```

THREAD POOLS

THREAD POOLS BASICS

Thread costs: performance + memory (stack per thread) → recycle threads for multiple tasks.
Tasks: potentially parallel work packages. Passive objects describing the functionality.
Thread Pool: Tasks queues. Smaller number of **working threads** grab tasks from queue and execute them. A task must run to completion before worker thread can grab a new one *except nested tasks/sub-tasks* .
Scalable Performance: *(Rule of thumb: # of Worker Threads = # processors + 1 (Pending I/O Calls))* Programs with tasks run **faster on parallel machines** → allows exploitation of parallelism **without thread costs**. Number of threads can be **adapted** to the system.
Advantages: Thread recycling *(save thread creation and release)* , **Higher level of abstraction** *(Disconnect task description from execution)* , **Number of threads configurable** on a per-system basis.
Limitations: Task must not wait for each other *(except sub-tasks)* would result in deadlock in Queue.
LIFO Queue: per worker thread and stealing is **FIFO**, Automatic degree of parallelism *(Default: As much worker threads as Processors)* , Optimized for recursive.
Fire-and-forget: Tasks may not finish (tasks are started without retrieving results). Async: `submit()` without `get()` oder CompletableFuture without `join()`. Task is run, but Exception will not get caught.

JAVA FORKJOINPOOL THREAD POOL

ForkJoinPool: Thread Pool for recursive Tasks (Divide and Conquer), work stealing, Subclass of ExecutorService
`ForkJoinPool.commonPool()`: static global Pool, singleton, used by `CompletableFuture`
Work Stealing: Jobs get submitted into the **global queue**, which distributes the jobs to the **local queues** of each worker thread. If one thread has no work left, it can **"steal" work from another threads** local queue instead of the global queue. This **distributes** the scheduling work over idle processors. Worker threads run as daemon threads (in TPL .NET too).

JAVA FORKJOINPOOL

`invoke(task)`: blocking
`invokeAll(task1, task2)`: blocking, creates fork and starts task
`submit(task)`: returns Future
`execute(task)`: async but does not return Future *fire and forget*
`task.fork()`: schedule async subtask
`future.get()`: wait for result/exception *prevent fire and forget by evaluating result* , can throw CancellationException, ExecutionException, InterruptedException
`result = task.join()`: wait and get result *prevent fire and forget*

```
// Task Launch, Async but without CompletableFuture
var threadPool = new ForkJoinPool();
Future<Integer> future = threadPool.submit(() → { // submit task into pool,
async
  int value = ...; /* long calculation */ return value;
});
T result = future.get(); // blocks until task terminated

var threadPool = new ForkJoinPool();
var left = threadPool.submit(() → countPrimes(2, INPUT_SIZE / 2));
var right = threadPool.submit(() → countPrimes(INPUT_SIZE / 2, INPUT_SIZE));
int result = left.get() + right.get();
```

Parallel Counting Example
What is a good choice for number of submitted tasks and why?: the number of available processor cores, because we want to be able to achieve maximum performance, but also not to create unnecessarily many tasks, which would also degrade performance

```
public class FlatParallelCount { // parallel counting
public static void main(String[] args) throws Exception {
    var threadPool = new ForkJoinPool();  // countPrimes returns int:
    Future<Integer> left = threadPool.submit(() → countPrimes(2, SIZE / 2));
    var right = threadPool.submit(() → countPrimes(SIZE / 2, SIZE));
    int result = left.get() + right.get();
} }
```

Recursive Action/Task
```
// Sequential:
int counter = 0; for (int n = 2; n < N; n++) { if (isPrime(n)) { counter++;}};
// Parallel and Recursive with RecursiveTask class:
class CountTask extends RecursiveTask<Integer> { // RecursiveAction: void
function
  private final int lower, upper;
  private static final int THRESHOLD = 1; // configurable
  public CountTask(int lower, int upper) {
    this.lower = lower;    this.upper = upper;
  }
  @Override
  protected Integer compute() { // TRESHOLD = avoid over-parallelizing
    if (lower == upper) { return 0; }
    if (lower + 1 == upper) {  return isPrime(lower) ? 1 : 0;  }
    if (upper - lower > THRESHOLD) { // parallel count
      int middle = (lower + upper) / 2;
      var left = new CountTask(lower, middle);
      var right = new CountTask(middle, upper);

      // V1 mit explizitem fork, blockiert noch nicht
      left.fork(); right.fork(); // fork = new task
      // V2 invokeAll, join danach ist sofort fertig
      invokeAll(left, right);

      return right.join() + left.join(); // join = wait for all threads
    } else { // sequential count:
      int count = 0;
      for (int number = lower; number < upper; number++) {
        if (isPrime(number)) { count++; }
      }
      return count;
    }
} }  // invoke is blocking:
Create explicit thread pool + invoke: var threadPool = new ForkJoinPool();
int result = threadPool.invoke(new CountTask(2, N));
int result = new CountTask(2, N).invoke(); // invokeAll() to start multiple tasks
```

THREAD POOL | PARALLEL FOR (.NET)

.NET Task Parallel Library (TPL)
Preferred way to write multi-threaded and parallel code. Provides public types and APIs in `System.Threading` and `System.Threading.Tasks` namespaces. **Efficient default thread pool** *(tasks are queued to the ThreadPool, supports algorithms to provide load balancing, tasks are lightweight)* , has **multiple abstraction layers** *(Task Parallelization: use tasks explicitly, Data Parallelization: use parallel statements and queries using tasks implicitly)* , Asynchronous Programming and PLINQ.

Parallel Statements in C#

Execute **independent** statements **potentially parallel** *(Start all tasks, implicit barrier at end)* .
```
Parallel.Invoke(
    () ⇒ MergeSort(l, m),
    () ⇒ MergeSort(m, r)
);
```

Execute **loop-bodies potentially parallel** *(Queue a task for each item, implicit barrier at the end)* .
```
Parallel.ForEach(list, file ⇒ Convert(file));
Parallel.For(0, array.Length,
    i ⇒ DoComputation(array[i]));
```

Parallel Loop Partitioning: Loop with lots of quickly executing bodies, inefficient to execute each iteration as parallel task → TPL **automatically groups multiple bodies** into a single task.

PLINQ (and Java Stream API)
LINQ: Set of technologies based on the integration of SQL-like query capabilities directly into C#.
PLINQ: Parallel LINQ implementation, simple, readable and parallel by creating segments from its data. Analog to Java Stream API.

```
// Get ISBN of all books with title «Concurrency»
from book in bookCollection.AsParallel() // parallel version
  where book.Title.Contains("Concurrency") select book.ISBN
// Map each number in a list to a bool value (isPrime=true):
from number in inputList.AsParallel().AsOrdered() // maintains order but slower
  select IsPrime(number)

// Java Stream API is implicitly ordered
bookCollection.parallelStream().unordered() // unordered explicit
.filter(book → book.getTitle().contains("Concurrency")) .map(book →
book.getISBN());
```

Thread Injection

TPL adds new worker threads **at runtime** every time a work item completes or every 500ms. **Maximize throughput** while using as **few threads** as possible. **Avoids deadlock** with task-dependencies *(but inefficiently since not designed for this. Deadlocks with `ThreadPool.SetMaxThreads()` are still possible)*. We should keep **parallel tasks short** to better profit from this automatic performance improvement.
Hill Climbing Algorithm: Measures throughput & varies number of worker threads

JAVA AND .NET | ANZAHL TASKS IM THREADPOOL

Wovon hängt die Anzahl Tasks bei beiden Verfahren ab?
.NET Parallel For: Anzahl freier Worker Threads
Java ForkJoinPool: Array-Länge und Threshold (bei RecursiveAction)

ASYNCHRONOUS PROGRAMMING

Unnecessary Synchrony: Blocking method calls are often used without need *(Long running calculations, I/O calls, database or file accesses)* . With an **asynchronous call**, other work can continue while waiting on the result of the long operation.
Models of Asynchronisms: Caller-centric *("pull", caller waits for the task end and gets the result, blocking call)* , **Callee-Centric** *("push", Task hands over the result directly to successor /follower task)*
Task Continuations: `Task.Run(task1) .ContinueWith(task2) .ContinueWith(task3) .Wait();`

C# Async Task

`var task = Task.Run(LongOperation); /* other work */ int result = task.Result;`
Multi-Continuation: `Task.WhenAll(task1, task2).ContinueWith(continuation);`
`Task.WhenAny(task1, task2).ContinueWith(continuation);`
(other threads get lost after first thread is done), Exceptions in fire&forget task get ignored
Exception Handling: `Wait()` or `TaskScheduler.UnobservedTaskException` (Registers for unobserved exceptions). *(Receives unhandled exceptions from fire&forget tasks)* . Should be executed as soon as task object is dead *(Garbage Collector)* .

Java Modern Async `CompletableFuture`

```
var future = CompletableFuture // tasks im commonPool
    .supplyAsync(() → longOP) /* returns CompletableFut. */
    .thenApplyAsync(v → 2 * v).thenAcceptAsync(v → ...println(v));
try { future.get(); } catch (ExecutionException e) {..}
```

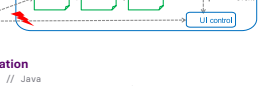
Async: `runAsync()` void Operation *(can be nested)* `supplyAsync()` returns value,
Continuation:
`thenAccept()` / `thenAcceptAsync()` void Op., `thenApply()` / `thenApplyAsync()` returns value,
Multi-Continuation:
`CompletableFuture.allOf(future1, future2)` and `CompletableFuture.any(...)`
`future.get()`: can throw CancellationException, ExecutionException, InterruptedException
Java `SwingUtilities.invokeLater(Runnable)`: Executed **asynchronously** on event dispatching thread. Should be used when an **application thread** needs to **update the GUI**.

Future<T> (implemented by CompletableFuture)
Represents **future result** *(asynchronous)* , Proxy for result.
`.submit()` submit task into pool + launch task, **`.get()`** waits for computation to complete and then retrieves result (throws `InterruptedException`, `ExecutionException`), **`.cancel(boolean mayInterruptIfRunning)`** Attempts to cancel task, but fails if already completed, cancelled or if running and argument false. Removes task from queue. `ExecutionException`

NON-BLOCKING GUI'S

Use case: If a UI is doing a long task, it should not freeze.
GUI Thread Model: Only **single-threading** *(Only a special UI-thread is allowed to access UI-components)* . The **UI thread** loops to process the **event queue**.

GUI Premise: No long operations in UI events, or else blocks UI. **No access to UI-elements by other threads** *(Exception in .NET & Android, Race Condition in Javas Swing)* .



Non-Blocking UI Implementation

```
// C# .NET
void buttonClick(...) {
  var url = textBox.Text;
  Task.Run(() ⇒ {
    var text = DownLoad(url);
    Dispatcher.InvokeAsync(
    () ⇒ {
      label.Content = text;
    });
  });
```

```
// Java
button.addActionListener(event →
    var url = textField.getText();
    CompletableFuture.runAsync(() → {
      var text = download(url); // to worker thread
      SwingUtilities.invokeLater(() → {
        textArea.setText(text); // to UI thread
      });
    });
)
```

C# ASYNC/AWAIT

```
// same UI code, but in C#
...
var url = textBox.Text;
var text = await
    DownLoadAsync(url);
label.Context = text;
...
```

```
async Task<string> DownloadAsync(string url) {
  var web = new HttpClient();
  string result = await web.GetStringAsync(url);
  return result;
}
```

async for methods: Caller may not be blocked during the entire execution of the async method.
await for tasks: "Non-blocking wait" on task-end / result.
Execution Model: Async methods run partly **synchronous** *(as long as there is no blocking await)* , partly **asynchronous** *(until the awaited task is complete)* .
Mechanism: Compiler dissects method into **segments** which are then executed completely synchronously or asynchronously.
Different Execution Scenarios: **Case 1:** Caller is a "normal" thread *(Usual case, Continuation is executed by a TPL-Worker-Thread)* , **Case 2:** Caller is a UI-thread *(Continuation is dispatched to the UI thread and processed by the UI-Thread as event)*
Async Return Value Types: `void` *("fire-and-forget")* , `Task` *(No return value, allows waiting for end)* , `Task<T>` *(For methods having return value of type T)* .
Async without await: Execute long running operation explicitly in task with `await Task.Run()`.

```
public async Task<bool> IsPrimeAsync(long number) {
  return await Task.Run(() ⇒ { // ohne diesen return nicht async
    for (long i = 2; i*i <= number; i++) {
      if (number % i == 0) { return false; }
    } return true;
  }); }
```

GPU (GRAPHICS PROCESSING UNIT)

End of Moores Law: We can no longer gain performance by "growing" sequential processors (and same price). Instead, **improve performance** by running code in **parallel** on **multi-core (CPUs)** *(Low Latency)* and many-core or **massively parallel co-processors (GPUs)** *(high throughput)* .
GPU's: specialized electronic circuits designed to accelerate the computation of **computer graphics**. They are faster than CPUs for suitable algorithms on large datasets. **Useful** for calculations which consist of **multiple independent sub-calculations**, not very useful for calculations where the results rely on the previous results *(like Fibonacci)* .
High Parallelization: A **CPU** offers few cores *(4, 8, 16, 64)* and is very fast. Programming is easier. A **GPU** offers a very large number of cores *(512, 1024, 3584, 5760)* and has very specific slower processors. It is optimized for throughput. Programming is more difficult.
GPU Structure: A GPU consists of multiple **Streaming Multiprocessors (SM)** which in turn consist of multiple **Streaming Processors (SP)** *(e.g. 1-30 SM, 8-192 SPs per SM)* .
SIMD: Single Instruction Multiple Data. The **same instruction** is executed simultaneously on **multiple cores** working on **different data elements** *(Vector parallelism)* . Saves fetch & decode instructions.
SISD: Single Instruction Single Data. Purely **sequential** calculations.
SIMT: Single Instruction Multiple Threads. The same instruction is executed in different threads over different data.

LATENCY VS. THROUGHPUT

Latency: **Elapsed time** of an event *(Walking from point A to B takes one minute, the latency is one minute)* .
Throughput: **The number of events** that can be executed per unit of time *(Bandwidth)*
There is a **tradeoff** between latency and throughput. Increasing throughput by pipelined processing, latency most often also increases. All pipeline stages must operate in **lockstep**. The **rate of processing** is determined by the **slowest step**.
Pipelining: Run processes in an overlapping manner.
Example: A program consists of two operations: Transfer data from CPU memory to GPU memory (T_1 units = $20ms$), Execute computation on the device (T_2 units = $60ms$).
What is the **latency** (non-pipelined)? $20 + 60 = 80\text{ms}$.
What is the **throughput** (pipelined)? Every $60ms$ an operation is finished.
Throughput = $1/60$ operations/ms.

CPUs	GPUs
– **Low latency** – Few but **optimized cores** – **General purpose** – Architecture and Compiler help to run any code fast	– Can execute **highly parallel data** operations – Simple but a **lot of cores** with cache per core – very useful for problems which consist of a **lot of independent data elements** – Efficiency must be achieved by **optimizing** the program
Aim: low latency per thread	**Aim:** high throughput

NUMA MODEL

NUMA stands for **Non-Uniform Memory Access**. CPUs on host and GPU devices each have local memories. There is **no main memory** between the two, so **explicit transfer** between CPU and GPU is needed. There is also **no garbage collector** on the GPU.

CUDA

Computer Unified Device Architecture. Is a **parallel computing platform and an API** for Nvidia GPU that allows the host program to use GPUs for general purpose processing.

CUDA Kernel
A kernel is a function that is executed n times in parallel by m different CUDA threads. Only the GPU knows when the task is finished.

CUDA CODE AND CUDA FLOW

```
// kernel definition on GPU
__global__
void VectorAddKernel(float *A, float *B, float *C) {
  int i = threadIdx.x; C[i] = A[i] + B[i];
}

void CudaVectorAdd(float* h_A, float* h_B, float* h_C, int N) {
  size_t size = N * sizeof(float);
  float *d_A, *d_B, *d_C; // for data on GPU:
  // 1. GPU memory allocate, error handling: handleCudaError(cudaMalloc(..))
  cudaMalloc(&d_A, size); cudaMalloc(&d_B, size); cudaMalloc(&d_C, size);
  // 2. Data transfer to GPU (HostToDevice):
  cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
  cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
  // 3. Kernel execution, N is amount of threads:
  VectorAddKernel<<<1, N>>>(d_A, d_B, d_C, N);
  // 4. Transfer results from GPU to CPU (DeviceToHost):
  cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
  // 5. Deallocate GPU memory:
  cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
}
```

PERFORMANCE METRICS

The performance is either limited by **memory bandwidth** or **computation**.
Compute Bound: Throughput is limited by calculation *(Cores are at the limit, but the memory bus could transfer more data)* .
This is better and reached if AI Kernel > AI GPU.
Memory Bound: Throughput is limited by data transfer *(Memory bus bandwidth is at its limit, but cores could process more data)* .
Arithmetic intensity: Defined as FLOPS *(Floating Point Operations per Second)* per Byte. The higher, the better.

$$\frac{\text{Number of operations}}{\text{Number of transferred bytes}} = \frac{\text{FLOPS}}{\text{Bytes}}$$

Compute Bound

$$t_{compute} > t_{memory}$$

$$\frac{\#ops}{BW_{compute}} > \frac{\#bytes}{BW_{memory}}$$

$$\frac{\#ops}{\#bytes} > \frac{BW_{compute}}{BW_{memory}}$$

Memory Bound

$$t_{compute} < t_{memory}$$

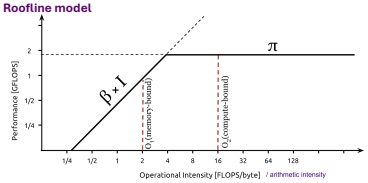
$$\frac{\#ops}{BW_{compute}} < \frac{\#bytes}{BW_{memory}}$$

$$\frac{\#ops}{\#bytes} < \frac{BW_{compute}}{BW_{memory}}$$

Exercise (Arithmetic intensity = operational intensity)
Assume 32-bit integer addition and multiplication. The components of z, x, and y are 4 bytes. Calculate the arithmetic intensity. x and y need to be fetched from memory and the result z is written back.
(Arrayzugriff = 4 Byte (pro int Wert). Operationen +,.., Zuweisung = ist keine Operation)*
1. `for(i=0; i<N, ++i) { z[i] = x[i] + y[i] * x[i]; }`
   Read x and y from memory, write z to memory. That's 2 reads and 1 write *(x is used twice but read only once)* . In case x, y and z are `int`s, we have $12\ (3 \cdot 4)$ bytes transferred and 2 arithmetic ops (+, *). The arithmetic intensity is $\frac{2}{12} = \frac{1}{6}$.
2. `for(i=0; i<N, ++i) { z[i] = x[i] + y[i]; }`
   1 Operation (Compute). 3 Zugriffe (access Memory) = $3 * 4 = 12$ Bytes.
   The arithmetic intensity is $\frac{1}{12}$

Maximum Threads = 1024: `VectorAddKernel<<<1, numElements>>>(d_A, d_B, d_C);`.

Roofline model



Provides performance estimates of a kernel running on differently sized architectures. Has three parameters: Peak performance, peak bandwidth vs. arithmetic intensity.
Peak performance is derived from benchmarking FLOPS or GFLOPS *(Giga-FLOPS, 10^9 FLOPS)* . The **peak bandwidth** from manuals of the memory subsystem. The **ridge point** is where the horizontal and diagonal lines meet = minimum Ar.Int. required to achieve the peak performance.

GPU ARCHITECTURE

Because there are so **many cores** on GPUs, it is possible to run many threads in parallel **without context switches**. This allows better parallelism without a performance penalty.

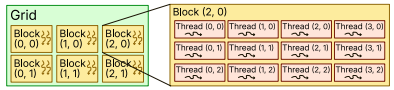
CUDA SIMT EXECUTION MODEL

Single **I**nstruction, **M**ultiple **T**hreads. The kernel is executed N times in parallel by N different CUDA threads.
Blocks: Threads are **grouped** in blocks. The host can define how many threads each block has *(up to 1024)* . Threads in one block can **interact** with each other but not with threads in other blocks.

CUDA KERNEL SPECIFICATION

Specifying Kernel: `VectorAddKernel<<<GRID_dimension, BLOCK_dimension>>>(A,B,C)`
Dimensions can be 1D, 2D or 3D and specified via `dim3` which is a structure designed for storing block and grid dimensions: `struct dim3{x; y; z};`
`VectorAddKernel<<<dimGrid, dimBlock>>>(A,B,C);`
Number of blocks in a grid: `dimGrid.x * dimGrid.y * dimGrid.z`
Number of threads in a block: `dimBlock.x * dimBlock.y * dimBlock.z`

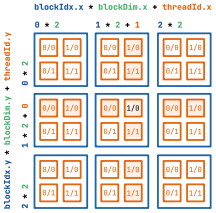
1D Grid: We can simply use integers. `VectorAddKernel<<<1, N>>>` creates 1 Block with N Threads.
2D Grid: `dim3 grid5(3,3); dim3 block5(5,5); VectorAddKernel<<<grid5, block5>>>`
3D Grid: `dim3 grid5(3,2,1); dim3 block5(4,3,1); VectorAddKernel<<<grid5, block5>>>`



Device Limits: **Max threads per block:** 1024, **Max thread dimensions per block:** (1024, 1024, 64)
Max grid size: (2'147'483'647, 65'535, 65'535)

Calculation Examples
`VectorAddKernel<<<dim3(8,4,2), dim3(16,16)>>>(d_A, d_B, d_C);`
Amount of Blocks: $8 \cdot 4 \cdot 2 = 64$
Amount of threads per block: $16 \cdot 16 = 256$
Threads in total: $64 \cdot 256 = 16'384$

If we have 1024 threads in a block, how many blocks are needed to launch N threads?
`int blocksPerGrid = (N + blockSize - 1) / blockSize;`
Rounding up is necessary because for 1025 threads, 2 blocks are required



Data Partitioning within threads

Data Access: Each kernel decides which data to work on. The programmers decide data partitioning scheme. `threadIdx.x/y/z` *(Thread no. in block)* , `blockId.x` *(Block no.)* , `blockDim.x` *(Block size)*
Partitioning in Blocks:

```
// kernel invocation
N = 4097; int blockSize = 1024;
int gridSize = (N + blockSize - 1) / blockSize;
VectorAddKernel<<<gridSize, blockSize>>>(A, B, C);
```

Boundary Check: More threads than necessary work on the data. If $N = 4097$, 5 Blocks with 1024 Threads are needed which results in **1023 unused threads**. Threads with $i \geq N$ must **not be allowed** to write to array C because they might **corrupt the working memory** of some other thread.

```
__global__
void VectorAddKernel(float *A, float *B, float *C) {
  // index based on (blockID, threadID).   1D/2D Grid
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < N) {  C[i] = A[i] + B[i];  } // boundary check
}
```

```
// Strided coalescing Beispiel:
// Kernel invocation for Array d_a (Pointer im Device Memory) with length len.
int N = len/2;
int blockSize = 1024; int gridSize = (N + blockSize - 1) / blockSize;
pairwise_sum<<<gridSize, blockSize>>>(d_a, len);

__global__ void pairwise_sum(int* array, int length) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (2*i + 1 < length) {
        array[2*i] += array[2*i + 1];
        array[2*i + 1] = 0;
    }  }
```

ERROR HANDLING

```
void handleCudaError(cudaError error) { if (error != cudaSuccess) {
        fprintf(stderr, "CUDA error: %s!\n", cudaGetErrorString(error));
        exit(EXIT_FAILURE);
} }
```

UNIFIED MEMORY

Unified memory allows automatic transfer from CPU to GPU and vice versa. No explicit Memory Copy (cudaMemCpy) needed, but other new rules. Error handling still needed.

CPU Code

```
A = (float*)malloc(size);// same for &B and &C:
vectorAdd(A, B, C, N);
free(A); free(B); free(C);
```

GPU Code

```
// same for &B and &C:
cudaMallocManaged(&A, size);
// data for A,B,C automatically
transferred to device (CPU)
VectorAddKernel<<<...,...>>>(A, B, C, N);
// wait for GPU, because not blocking:
cudaDeviceSynchronize();
cudaFree(A); cudaFree(B); cudaFree(C);
```

GPU PERFORMANCE OPTIMIZATIONS

Hardware: A scalable array of multithreaded **Streaming Multiprocessors** (SMs), the **threads** of a thread block execute **concurrently** on one multiprocessor, multiple **thread blocks** can execute **concurrently** on one multiprocessor. When thread blocks **terminate**, new blocks are launched on the free multiprocessors.

MATRIX ADDITION

```
__global__
void MatrixAddKernel(float *A, float *B, float *C) {
  int column = blockIdx.x * blockDim.x + threadIdx.x;
  int row = blockIdx.y * blockDim.y + threadIdx.y;
  if (row < A_ROWS && col < A_COLS) { // boundary checking
    C[row * A_COLS + col] = A[row * A_COLS + col] + B[row * A_COLS + col];
} }
const int A_COLS, B_COLS, C_COLS = 6;
const int A_ROWS, B_ROWS, C_ROWS = 4;
dim3 block = (2,2); dim3 grid = (3,2); // (3,2) == (3,2,1)
MatrixAddKernel<<<grid,block>>>(A,B,C);
```

MATRIX MULTIPLICATION

Parallelization: Every thread computes one element of the result matrix C. Can be parallelized because results do not depend on each other.

```
__global__
void MatrixMultiply(float *A, float *B, float *C) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  int j = blockIdx.y * blockDim.y + threadIdx.y;
  if (i < N && j < M) { // boundary checking
    float sum = 0;
    for (int k = 0; k < K; k++) {
      sum += A[i * K + k] * B[k * M + j];
    }
    C[i * M + j] = sum;
} }
```

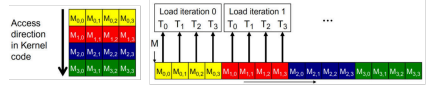
MAPPING THREADS / BLOCKS TO GPU WARPS

Warps: Blocks are split into **warps** *(1 Warp = 32 Threads)* and all threads within execute the same code. If there aren't enough threads to fill a warp, "empty" threads are launched. A number of warps constitutes a **thread block**. A number of **thread blocks** are assigned to a **Streaming Multiprocessor**. The whole GPU consists of several SM.
Thread blocks are scheduled in **parallel** or **sequentially**. Once a thread block is **launched** on a SM, **all of its warps are resident** until their execution finishes. Therefore, a **new block on a SM** is **not launched until** there is a **sufficient number** of free registers and shared memory for **all warps** of the new block.

Warp Execution: All threads in a warp execute the same instruction *(SIMD)* . A SM can accommodate all warps of a block, but only **a subset** is **running in parallel** at the same time *(1 to 24)* .
Warps are split: sequentially by threadIdx.x/y/z in 32 steps
Divergence: Different branches in same warp. *if/switch/while/do/for* . Performance problem, because SM can execute only instruction of one branch and the other threads have to wait.

```
// bad case, divergence in same warp  // good case, all t in warp in same branch
if (threadIdx.x > 1) { } else { }   if (threadIdx.x / 32 > 1) { } else { }
```

DRAM (Dynamic Random Access Memory): **Global memory of a CUDA device** is implemented with DRAMs. If a GPU kernel accesses data from **consecutive locations**, the DRAMs can supply the data at a much **higher rate** than if a random sequence of locations were accessed.
Memory Coalescing: Improve performance by using Cache of DRAM. If threads in warp **simultaneously** access **consecutive memory locations** (32 byte areas), their reads can be combined into a single access **(burst)**.
Coalesced Accesses: All threads should Read/Write from same burst section in warp, but doesn't matter if only individual elements of burst or swapped access in burst.
Not Coalesced Accesses (strided): Avoid inperformant read/write action over different warp bursts.
Use Coalescing access as follows: `data[(Expression without threadIdx.x) + threadIdx.x]`
Coalescing with Matrices: Matrices get linearized to a 1D array. The row of the matrix should be the longer side so that there are as many coalescing accesses as possible.



MEMORY MODEL

All threads have the access to the same **global memory**. Each thread block has **shared memory** visible to all threads of the block and with the same lifetime as the block *(Has higher bandwidth and lower latency than local or global memory but longer latency and lower bandwidth than registers which are private to a thread)* . Each thread has **private local memory** *(in device memory, high latency and low bandwidth, same as global)* . Constant, texture and surface memory also reside in device memory.
Memory Hierarchy: **Shared Memory** *(per SM, fast, shared between threads in 1 block, a few KB, __shared__ float x)*, **Global Memory** *("Main Memory" in GPU Device, slow, accessible to all threads, in GB, cudaMalloc())* **Registers** *(private to a thread, fastest but very limited storage)*
Constant memory: Constant variables are stored in the **global memory** but are **cached**.
Shared Memory Declaration: With keyword `__shared__`. A **static array size** is necessary. Limited memory, around 48KB. Multidimensionality is allowed.
Fast Matrix Multiplication: By **reducing global memory traffic**. Partition data into subsets called tiles which fit into shared memory *(the row & column that should be multiplied and the result cell)* . The kernel computation on these tiles must be able to run **independently** of each other. Because the shared memory is **limited**, load the tiles in **several steps** and calculate the **intermediate result** from this.

```
__global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int Width) {
  __shared__ float Mds[TILE_WIDTH][TILE_WIDTH];
  __shared__ float Nds[TILE_WIDTH][TILE_WIDTH];
  int bx = blockIdx.x; int by = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;
  // identify row and column of the d_P element to work on
  int Row = by * TILE_WIDTH + ty;   int Col = bx * TILE_WIDTH + tx;
  float Pvalue = 0;
  // loop over d_M and d_N tiles required to compute d_P element
  for (int m = 0; m < Width/TILE_WIDTH; ++m) {
    // collaborative loading of d_M and d_N tiles into shared memory
    Mds[ty][tx] = d_M[Row*Width + m*TILE_WIDTH + tx];
    Nds[ty][tx] = d_N[(m*TILE_WIDTH + ty)*Width + Col];
    __syncthreads(); // CUDA equivalent to wait()
    for (int k = 0; k < TILE_WIDTH; ++k) {
      Pvalue += Mds[ty][k] * Nds[k][tx];
    }
    __syncthreads();
  }
  d_P[Row*Width + Col] = Pvalue;
}
```

__syncThreads() is only allowed in if/else if all threads of a block choose the same branch, otherwise undefined behavior.

HIGH PERFORMANCE COMPUTING (HPC) CLUSTER PARALLELIZ.

Cluster programming is the **highest possible parallel acceleration** *(Factor 100 and more)* . Used for **general purpose programming**, lots of CPU cores. Combination of CPUs and GPUs possible.

Computer Cluster: Network of **powerful** computing nodes, firmly connected at one location. Very **fast interconnect** *(like 100GBit/s)* , used for big simulations *(Fluids, Weather, Traffic, etc.)*
SPMD: This is the mostly used programming model, "high level".
Single Program *(All tasks execute their copy of the same program simultaneously)* , **Multiple Data** *(all tasks may use different data)* . (with MPI below)
MPMD: Also a "high level" programming model. **Multiple Program** *(Tasks may execute different programs simultaneously)* , **Multiple Data** *(all tasks may use different data)* .
Hybrid Memory Model: All processors in a machine can **share** the memory. They also can **request** data from other computers. *(non-uniform memory access: not all accesses take the same time)*.
Message Passing Interface (MPI): Distributed programming model. Is a choice for Parallelization on a cluster, Industry-Standard libraries for multiple programming languages.
MPI Model: Notion of processes *(Process is the running program plus it's data)* , parallelism is achieved by running **multiple processes**, co-operating on the **same** task. Each process has **direct access** only to its **own data** *(variables are private)* . Inter-Process-Communication by sending and receiving messages.
SPMD in MPI: All processes run their **own local copy** of the program & data. Each process has a **unique identifier** (rank), processes can take **different paths** through the program depending on their IDs. All processes start and terminate synchronously. Synchronization is done with barriers. Usually, **one process per core** is used *(to maximize the benefit of parallelization)* .
Formalizing Message: A message transfers a number of data items from the memory of one process to the memory of another process *(Typically contains ID of sender and receiver, data type to be sent, number of data items, data itself, message type identifier)* .
Communication modes: **Point to Point** *(very simple, one sender and one receiver. Relies on matching send and receive calls)* and **Collective communications** *(between groups of processes. **Broadcast:** one to all, **Scatter:** Split data and send each chunk to different node, **Gather:** Collect the chunks back at the originating node)* .
Process Identification: **Rank** = number within a group, incremental numbering from 0.
Unique Identification = (Rank, Communicator)
Send and Receive:

```
MPI_Send(void * data, int count, MPI_Datatype datatype, int destination, int tag,
MPI_Comm communicator) // tag: freely selectable number for msg type (≥ 0)
MPI_Recv(void * data, int count, MPI_Datatype datatype, int source, int tag,
MPI_Comm communicator, MPI_Status* status) // status: error information
```

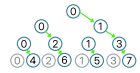
Example direct communication:

```
MPI_Send(&value, 1, MPI_INT, receiverRank, tag, MPI_COMM_WORLD);
MPI_Recv(&value, 1, MPI_INT, senderRank, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Array send: int array[LENGTH];

```
MPI_Send(array, LENGTH, MPI_INT, receiverRank, tag, MPI_COMM_WORLD);
MPI_Recv(array, LENGTH, MPI_INT, senderRank, tag, MPI_COMM_WORLD, MPI_STATUS_IGN);
```

MPI_Bcast:
Efficient, because root node does **not send the signal individually** to each node, the **other nodes help** spread the message to others: `MPI_Bcast(void * data, int count, MPI_Datatype datatype, int root, MPI_COMM_WORLD)`



MPI_Reduce: Reduction is a classic concept: reducing a set of numbers into a smaller set of numbers via a function *(e.g. [1,2,3,4,5] ⇒ sum ⇒ 15)* . Each process contains one integer, `MPI_Reduce` is called with a root process of 0 and using `MPI_SUM` as the reduction operation. The four numbers are added and stored on the root process. Job is done in a **distributed manner**.

```
MPI_Reduce(void* send_data, void* recv_data, int count, MPI_Datatype datatype,
MPI_Op op, int root, MPI_COMM_WORLD)
```

*send_data: array of elements of type datatype to reduce from each process, recv_data: relevant on the root process. contains the reduced result and has a size of sizeof(datatype) * count.*
MPI_OP: `MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD` (multiplies all), `MPI_BAND/MPI_LAND` (Bitwise/Logical AND), `MPI_LOR` (Logical OR), `MPI_MAXLOC` (Same as max plus rank of process that owns it)
MPI_AllReduce: Many parallel applications require accessing the reduced results **across all processes**. This function reduces the values and distributes the result to all processes. Does not need a root node. This is an **implicit broadcast** to all processes.

```
MPI_Allreduce(void* send_data, void* recv_data, int count, MPI_Datatype datatype,
MPI_Op op, MPI_COMM comm)
```

MPI_Gather: Gather together multiple values from different processors.

```
MPI_Gather(&input_value, 1, MPI_INT, &output_array, 1, MPI_INT, 0, MPI_COMM_WORLD)
```

MPI BOILERPLATE CODE

```
int main(int argc, char * argv[]) {
  // first MPI call, broadcasts to all processes, communicator formed, ..
  MPI_Init(&argc, &argv);
  int rank; int size; int len;
  MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Process Identification by rank
  MPI_Comm_size(MPI_COMM_WORLD, &size); // number of processes in comm.

  char name[MPI_MAX_PROCESSOR_NAME]; MPI_Get_processor_name(name, &len);
  printf("MPI process %i on %s\n", rank, name);

  MPI_Barrier(MPI_COMM_WORLD); // blocks/waits for processes in communicator

  MPI_Finalize(); // MPI Finalization, clean up
}
```

APPROXIMATION OF π VIA MONTE CARLO SIMULATION

Draw a circle inside of a square and randomly place dots in the square. The ratio of dots inside the circle to the total number of dots will approximately equal $\pi/4$.

```
// Parallel, the trials are split
across different nodes
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
// each process recv. a different seed:
srand(rank * 4711);
// Each process computes a subtask:
long hits = count_hits(TRIALS / size);
long total;
MPI_Reduce(&hits, &total, 1, MPI_LONG,
  MPI_SUM, 0, MPI_COMM_WORLD);
if (rank == 0) {
  double pi = 4 * ((double)total
                   / TRIALS);
  printf("Pi approx. %f", pi);
}
```

```
// Sequential
long count_hits(long trials) {
hits = 0, i;
  for (i = 0; i < trials; i++) {
    double x = (double)rand()/RAND_MAX;
    double y = (double)rand()/RAND_MAX;
    // distance to center bigger than
radius=1 :
    if (x * x + y * y ≤ 1) { hits++;}
  }
  return hits; }
```

OPENMP

Node: A standalone **"computer in a box"**. Usually comprised of multiple CPU/processors/cores, memory, network interfaces etc. Nodes are **networked together** to comprise a **supercomputer**. Each node consists of 20 cores. The processes do **not share memory**, they must use messages.
Threads: Default are 24 on a single Node in OST cluster. Can be set with `omp_set_num_threads()` or with the `OMP_NUM_THREADS` environment variable. Threads range from 0 *(master thread)* to N-1.
HPC Hybrid memory model: Run a program on multiple nodes. No Shared Memory *(NUMA)* between Nodes. Shared Memory *(SMP)* for Cores inside a Node.
OpenMP: Is a programming model for different languages. **Allows to run multiple threads**, distribute work using synchronization and reduction constructs. **Shared Memory** *(shared memory process consists of multiple threads)* , **Explicit Parallelism** *(Programmer has full control over parallelization)* and **Compiler Directives** *(Most OpenMP parallelism is specified through the use of compiler directives (pragmas) in the source code)* .

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char* argv[]) {

    const int np = omp_get_max_threads(); // executed by initial thread
    printf("OpenMP with threads %d\n", np); // executed by initial thread
    #pragma omp parallel // pragma spawns multiple threads (fork)
    {
        const int np = omp_get_num_threads(); // executed in parallel < v
        printf("Hello from thread %d\n", omp_get_thread_num());
    } // thread order not fixed. after execution, threads synchronize & terminate

    // for loop, iteration variable i is implicitly private during loop
    #pragma omp parallel for
        for (i=0; i<n; i++) { ... }

    // memory model:
    int A, B, C // automatically global because outside of pragma
    #pragma omp parallel for private(A) shared(B) firstprivate(C)
        for(...)
        // private copy of A per thread,
        // C is private but gets initial value from global C.
        // after loop is over, threads die + A and B cleared/removed from memory

    int A; // setting A private:
    #pragma omp parallel for private (A)
        for (...) // is equal to:
    #pragma omp parallel
        int A = 0 // automatically private because inside of pragma
    #pragma omp for ...
        for (...)

    // Mutex (avoid Race conditions)
    // performance overhead, single threading is faster than this:
    int sum = 0;
    #pragma omp parallel for
        for (int i = 0; i < n; i++)
    #pragma omp critical { sum += i; } // only one thread at a time

    // Lightweight Mutex (Atomic)
    int sum = 0; int i;
    #pragma omp parallel for
        for (i = 0; i < n; i++)
        #pragma omp atomic { sum += i }
}
```

For Loop: OpenMP handles oversubscription (too many threads).
Reduction across threads: When using `reduction(operator: variable)`, a **copy** of the reduction variable per thread is created, initialized to the identity of the reduction operator *(+ = 0, * = 1)* . Each thread will then **reduce** into its local variable. At the end of the `parallel` region, the local results are **combined into the global variable**. Only associative operators allowed *(+,* not −, ÷*)* .

```
// Code using the reduction clause
int sum = 0;
#pragma omp parallel for reduction(+: sum)
    for (int i = 0; i < n; i++) { sum += i; }

// The same code without the reduction clause
int sum = 0;
#pragma omp parallel {
  int intermediate_sum = 0; // private
  #pragma omp for
    for (int i = 0; i < n; i++) { intermediate_sum += i; } // thread partial sum
  #pragma omp atomic // reduction is protected with atomic
    final_sum += intermediate_sum; }
```

Hybrid: OpenMP + MPI

```
int numprocs, rank; int iam = 0, np = 1;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &numprocs); MPI_Comm_rank(MPI_COMM_WORLD, &rank);
#pragma omp parallel default(shared) private(iam, np) {
  np = omp_get_num_threads(); iam = omp_get_thread_num();
  printf("I am T %d out of %d from P %d out of %d\n", iam, np, rank, numprocs);
} MPI_Finalize();
```

Sequential count_hits to approximate π with Monte Carlo Simulation in OpenMP

```
long count_hits(long trials) {
  long hits = 0; long i; double x,y;
  #pragma omp parallel {
    #pragma omp for reduction(+:hits) private(x,y)
      for (i = 0; i < trials; i++) {
        double x = random_double(); double y = random_double();
        if (x * x + y * y ≤ 1) { hits +; }
    } /* for */ } /* parallel */
  return hits;
}
```

PERFORMANCE SCALING

Difficulties with parallel programs: Finding parallelism, granularity of a parallel task, moving data is expensive, load balancing, coordination & synchronization, performance debugging.
Scalability: The ability of hard- and software to deliver **greater computational power** when the number of **resources is increased**.
Scalability Testing: Measuring the ability of an application to perform well or better with varying problem sizes and numbers of processors (strong scaling/weak scaling). Does **not** test the applications **general functionality** or correctness.
Primary challenge of parallel computing is **deciding how best to break up a problem** into individual pieces that can be computed separately. It is **impractical** to develop and test large applications using the **full problem size**. The problem and number of processors are **scaled down** at first.

STRONG SCALING (CPU BOUND APPLICATIONS)

The number of processors is increased while the problem size remains constant. Results in a reduced workload per processor. Mostly used for long running CPU bound applications.
Amdahls Law: The speedup is **limited by the fraction of the serial part** of the software that is not amenable to parallelization. Sweet spot needs to be found. It is reasonable to use **small amounts of resources for small problems** and **larger quantities of resources for big problems.**
T = total time, p = part of the program that can be parallelized., N = amount of processors
$T = (1-p)T + \frac{T_p}{N} = T_s + T_p$
$T_N = T_p/N + (1-p)T$, Speedup $\leq 1/(s + p/N)$, Efficiency $= T/(N \cdot T_N)$
Amdahls law **ignores the parallel overhead**. Because of that, it is **the upper limit** of speedup for a problem of fixed size. This seems to be a **bottleneck** for parallel computing.

Examples:
90% of the computation can run parallel, max speedup with 8 processors?
$1/(0.1 + 0.9/8) \approx 4.7$

25% of the computation must be serial. Max speedup with ∞ Processors?
$1/(0.25 + 0.75/\infty) \approx 1/0.25 = 4$

To gain a $500\times$ speedup on 1000 processors, Amdahls law requires that the proportion of serial part cannot exceed what?
$500 = 1/(s + \frac{1-s}{1000}) \Rightarrow s + \frac{1-s}{1000} = \frac{1}{500}$

$\Rightarrow 1000s + (1-s) = 2 \Rightarrow 999s = 1 \Rightarrow s = \frac{1}{999} \approx 0.1\%$

WEAK SCALING (MEMORY BOUND APPLICATIONS)

The number of processors and the problem size is increased. Mostly used for large memory-bound applications where the required memory cannot be satisfied by a single node.
Gustafson's law: Based on the approximations that the **parallel part scales linearly** with the amount of resources, and that the **serial part** does **not increase** with respect to the size of the problem. **Speedup** $= s + p \cdot N = s + (1-s) \cdot N$
In this case, the problem size assigned to each processing element stays **constant** and **additional elements** are used to solve a **larger total problem**. Therefore, this type of measurement is justification for programs that take a lot of memory or other system resources.

Example:
64 Processors. 5% of the program is serial, Scaled weak speedup?
$0.05 + 0.95 \cdot 64 = 60.85$

CLI COMMANDS

```
mpicc -o hello hello.c
sbatch script.sub
# with slurm for workload distribution, result as slurm file
#!/usr/bin/env bash
#SBATCH --job-name=compare
#SBATCH --time=01:00:00
#SBATCH --ntasks=60
#SBATCH --nodes=3
#SBATCH --partition=nodes
module purge
module load gcc-toolchain openmpi # -n = number of processes
mpiexec -n $SLURM_NTASKS --mca btl self,vader ./hello
```

Compilation & Execution: `mpicc HelloCluster.c && mpiexec -c 24 a.out && sbatch -hi.sub`